Problem-Sized Lean Worlds

An authority-separated architecture from AI search to public mathematical claims

Will Cook*

August 2026

Abstract

This paper presents an executable architecture for AI-assisted mathematics under proof abundance. It separates six things that are commonly collapsed: cognition authority, permission to change shared state, validation capacity, evidence class, public-claim authority, and reviewer attention. Work advances only when the next gate matches the evidence. A failed route may change later search or validation without being relabelled as a theorem.

The architecture organises work around *problem-sized mathematical worlds*, not isolated theorem files. Each world joins formal declarations, experiments, source literature, failed mechanisms, open obligations, and a reviewed public boundary. Formal dependencies, authored mathematical meaning, and public claims remain separate graphs, so a dense connection cannot make a claim stronger. An agent enters a world through a compiled comprehension packet: it federates the corpora by content digest, states the endpoint and claim ceiling before any route, separates proved results from open producers and closed routes, and declares both what it omitted and when it exceeded its context budget. The contribution is an implemented architecture that connects research state, scoped changes, formal checks, and public explanation while keeping their evidential roles distinct; the related-work section compares it with neighbouring systems along stated dimensions and claims no priority.

The mathematical workflow keeps conjecture, computation, formal proof, interpretation, public wording, and external review distinct. Computation may reject a route or supply finite evidence; it does not prove an unbounded theorem. Lean verifies that a proof establishes the formal statement written in the source; it does not verify whether that statement captures the intended mathematics or whether the paper describes it well. Comparator checks selected propositions and their axiom boundary; a local review-selection layer prepares result families for external registration and review. Neither decides novelty, importance, acceptance, or whether an open problem has been solved.

The public repository is a self-contained output whose Lean source, claim record, papers, navigation, and release checks can be inspected from a fresh clone. A worked example follows a finite certificate for Erdős Problem 249 and shows why a checked finite range cannot become an unbounded conclusion. This is a working prototype rather than a validated multi-user service. As of 31 August 2026, the author had not recorded a completed external cold-clone use or an accepted external contribution. The rule is simple: each stage may pass forward only the claim its evidence supports, together with the boundary it does not cross. All eight problems remain open. The paper does not claim a solution to any of them.

Main contribution and claim ceiling

Contribution. The prototype keeps six authorities separate and makes every transition from conjecture to public wording carry its evidence and its limit. Problem-sized worlds join formal dependencies, mathematical interpretation, failed routes, and reviewed claims without treating any one graph as authority for the others. **Demonstration.** The mechanism is exercised on a self-contained public Lean corpus and an audited finite certificate for Erdős Problem 249. **Ceiling.** No open problem is claimed solved, and external use remains untested.

Authorship and AI use. Will Cook built and directed the research infrastructure and reviewed the claims when he could. AI agents did most of the research and drafting. Cook did not independently verify every claim.

1 The problem: many kinds of evidence

Suppose an AI system proposes a proof of a mathematical statement. Several different questions immediately arise.

1. Did the system understand the intended problem?
2. Did it explore the important alternatives and notice counterexamples?
3. Does the proposed formal statement say what the informal statement says?
4. Does Lean accept a proof of that exact formal statement?
5. Does the public explanation stay within the proved scope?
6. Has anyone outside the producing system reviewed, accepted, or absorbed the result?

No single test answers all six. A numerical experiment can expose a false conjecture without proving a universal one. Lean can certify a theorem while remaining silent about the prose around it. A release checker can preserve a reviewed sentence without making the review correct. External attention can signal interest without validating a proof. The architecture exists to keep these questions separate while allowing work to move between them.

The public repository studied here contains Lean source, papers, and release machinery around eight open Erdős problems. At this revision all eight problems remain open. The repository contains substantial intermediate theorems, exact reformulations, conditional reductions, finite certificates, and no-go results. The surrounding private workbench is broader: it supports research, agent coordination, computational experiments, formalisation, exposition, and controlled public projection. Private machinery explains how the work was produced; it supplies no hidden proof authority to the public clone.

The novelty claim and its ceiling. This paper does not claim to have invented agents, queues, file locks, theorem graphs, Lean checking, pull requests, or credit records. Its candidate contribution is an executable *claim-transition architecture*. It models reasoning, mutation, validation, interpretation, publication, and review as separately scarce and separately authorised operations. The operations are recombined only at explicit fan-in gates. The same machinery carries proofs, counterexamples, no-gos, and unresolved obligations while preserving their different evidence classes. Crucially, a local failure may alter the next agent's route, context, lease, experiment, or validator, but never the truth status of a mathematical statement.

Four status words are used deliberately. *Implemented* means source and an executable check or receipt exist. *Projection* means a generated view over more authoritative records. *Inactive* means implemented machinery was not running at the reported snapshot. *Hypothesis* means a proposed experiment, not a reported capability. Thus the router, work leases, corpus maps, Lean gates, Comparator, review-selection records, and release checks are implemented; dashboards and graph views are projections; the resident maintenance daemon was inactive at one recorded snapshot; and mass frontier-model mining or training on the no-go graph remain hypotheses.

1.1 Two architectural claims

A problem is a mathematical world, not a folder. For an agent, the useful unit is neither the repository nor one theorem. It is a bounded neighbourhood inside a problem-sized world. The neighbourhood begins with the exact endpoint and current claim ceiling, then selects established premises, nearby declarations, open producers, consumers, alternative coordinates, executable experiments, scoped falsifiers and no-gos, source literature, and public boundaries. Every returned edge says why it exists and what authority it lacks; the packet says what it omitted and where to expand. This makes deep corpora navigable without converting file proximity, lexical similarity, or model interpretation into proof.

Resource	Governing object	What possession cannot imply
Reasoning scope	Task-conditioned context and trace	Mutation permission or mathematical truth
Mutation permission	Exact path/work lease and change	Formal acceptance or public promotion
Validator capacity	Single-flight slot and build receipt	Theorem failure when a request is deferred
Evidence class	Experiment, counterexample, theorem, or authored relation	Automatic promotion into another class
Public-claim permission	Reviewed claim record and release relationship	Novelty, independent review, or acceptance
Reviewer attention	Ranked packet and recorded human outcome	Proof authority or canonical status

Table 1: The six non-fungible resources of the claim-transition architecture.

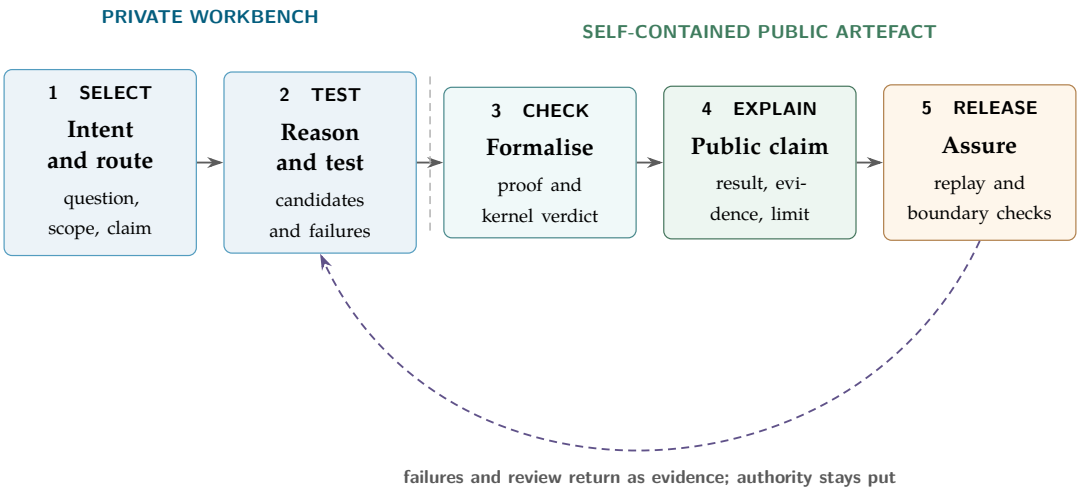

Figure 1: The end-to-end architecture. The dashed vertical line marks the release boundary. Everything to its right remains usable without the private workbench.

Authority is separated before it is recombined. The architecture treats six resources as non-fungible. More reasoning cannot buy a write lease; a write cannot buy a Lean receipt; a Lean receipt cannot buy semantic correctness; approved wording cannot buy novelty or community acceptance. They meet only through typed artefacts at an explicit gate.

There are consequently two coupled graphs with a guarded crossing. Mathematical evidence changes the mathematical graph: theorem, counterexample, no-go, or bounded experiment, each with its own class. Operational evidence changes the control graph: a route miss, stale view, repeated workaround, validation failure, or resource conflict may justify a new router, skill, check, or standard after a generalisation guard. Reviewed claim and publication artefacts connect the two. Neither graph may rewrite the other by implication.

2 The whole lifecycle in one picture

Figure 1 shows the main path. It is a loop rather than a one-way publishing pipeline: failed proofs, reviewer objections, and public drift can all return a precise lesson to an earlier stage. What may move backward is information about an error or a better method. What may not move backward is authority: a paper cannot make an unproved statement true, and an external review route cannot alter what Lean checked.

The lifecycle has four recurring operations. First, *selection*: choose a bounded object rather than treating the whole repository as one prompt. Second, *execution*: run an experiment, edit a proof, or write an

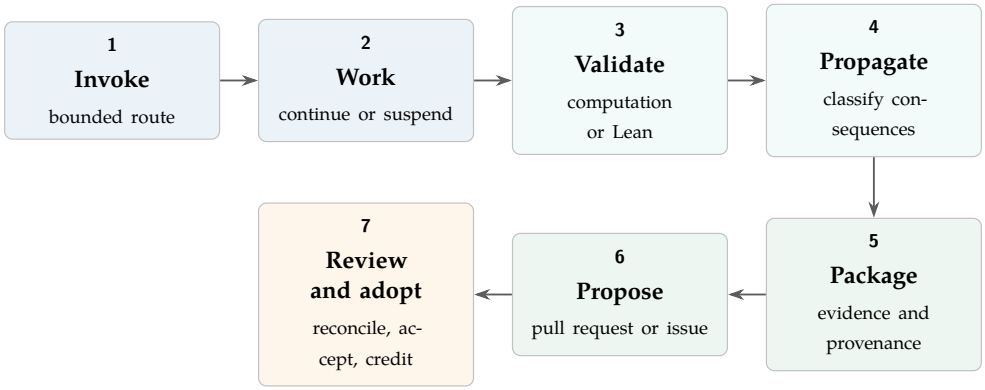

Figure 2: The clone-local job lifecycle. Closure means that the present delta has evidence and dispositions; it does not mean that the open problem is solved.

explanation. Third, *validation*: ask the authority appropriate to that object. Fourth, *binding*: record the result, limitation, and source so a later agent does not have to rediscover them. The final feedback step turns a local success or failure into a reusable route, check, or warning when it genuinely generalises.

The public `scripts/proof_cockpit.py` card gives an agent this separation. It reads checkout and toolchain identity, corpus scale, registered open propositions, problem obligations, and workbench sessions from public files. It imports no private state and adds no evidence layer to Figure **??**. Its `--check` mode runs the claim-registry, cold-clone, and projection-freshness checks; neither mode invokes Lean or decides whether prose follows from a theorem.

Once a person has compared a formal theorem with its public wording, a program can preserve the resulting decision about names, files, wording, and limits. It cannot decide whether the decision was correct. The workflow does not technically force a second independent mathematician: one maintainer can edit the Lean statement, record, and prose together so that every comparison agrees with the same mistake.

2.1 Skill-addressable job lifecycles

The public clone gives each recurring job one entry skill. A skill owns the job's starting state, permitted mutations, required evidence, stopping or re-entry condition, and next owner; it does not acquire the authority of the objects it coordinates. Figure 2 records the ordinary research path.

The current owners are `explain-public-system` for reader orientation, `run-coupled-research-goals` for discovery–stewardship coordination, `mine-open-problem` for invocation and research, `lean-concurrent-validation` for bounded Lean checks, `propagate-research-consequences` for downstream reconciliation, `erdos-research-return` for packaging and maintainer assimilation, and `submit-pull-request` for preparing the Git proposal. The separate `add-open-problem` lifecycle moves through proposal, incubating formal lane, and fully indexed public world. `public-mathematical-writing` governs a paper projection when propagation reaches exposition. These names are repository entrypoints rather than a second claim taxonomy.

3 The private workbench

3.1 Disk is shared memory

The private system does not treat a chat transcript as the project. Durable state lives in files: source code, structured records, append-only events, generated views, validation receipts, and authored explanations. A conversation may propose a change, but the file and its governing checker determine whether the change exists. This makes work resumable across agents and time: the next agent reads a bounded current-state packet and the relevant sources rather than trusting a summary of an earlier conversation.

Different file classes carry different authority. Raw operator language preserves what was asked. Work records say what has been claimed and by whom. Source files contain implementations and proofs.

Receipts record what actually ran. Authored papers explain; generated indexes help readers navigate. A generated projection is therefore a map of the territory, not the territory's source of truth.

At entry, a deterministic router selects a small context packet for the task. The packet gives candidate objects, authority boundaries, relevant standards, and legal next actions. This "small map before large source" rule reduces blind search without pretending that a summary is proof.

3.2 Type A and Type B

The system uses *Type A* and *Type B* to describe substrate access, not model quality.

Actor	What it can do	Governing limit
Type A	Inspect live project state; use repository tools; edit claimed files; run tests; bind receipts and status.	It may change only the substrate and paths its task authorises, and its claims remain limited by the relevant validator or human review.
Type B	Reason over a selected packet through an external model, web, API, or operator-carried exchange; return research, critique, or a candidate.	It has no direct private-substrate authority. A Type A actor or the operator must check and apply useful output.

Table 2: Type A/B is an authority distinction. Either type may be weak or strong; a delegated tool-using agent is still Type A if it has live substrate access.

This separation has two benefits. External reasoning can be used without pretending it observed private state, and local mutation stays attached to tests, paths, and receipts. It also prevents a common category error: calling every delegated worker Type B, or treating Type B as a cheaper or less capable kind of intelligence.

3.3 Concurrency without shared-state confusion

Several agents may work at once, but concurrency is admitted only where the write scopes can be separated. A work item identifies the objective and expected evidence. Before mutation, an agent claims exact paths for a bounded lease. Independent claims may proceed concurrently; overlapping claims must be coordinated or deferred. Fan-out is followed by a fan-in barrier where the results are compared, validated, and integrated.

Coordination state is written to append-only ledgers and immutable receipts, not inferred from who last spoke in chat. Generated status pages are read models over those records. This gives three distinct facts: an agent may be running, its proposed change may exist, and its change may have passed the owner's gate. None implies the next. Bounded job counts and focused Lean builds also prevent concurrency from exhausting the machine or making two builds corrupt each other's evidence.

3.4 The control plane is executable

The router projects a typed option surface over skills, standards, paper modules, live work, and source artefacts, then selects a task-conditioned packet before large sources are opened. A cold agent can move from a one-line flag to a short card, a working context, and finally the authority-bearing file. Each compression layer carries its source, safe drill-down, and authority posture. Context selection is therefore an inspectable operation rather than an invisible choice by the language model.

The work ledger separates past work from present permission. Completed, reopened, and superseded work are append-only lifecycle events. Live claims are short leases over exact paths or work objects. Directory claims collide with their children; expired leases lose authority unconditionally; a crashed holder leaves an expiry and dirty-handoff record rather than silently disappearing. A generated cohort view may report activity, collisions, and unknown scope, but it cannot grant a write. Thus "an

agent mentioned this file", "an agent holds this file", and "a change to this file was accepted" remain different statements.

An additional resident-runtime layer, called metabolism, turns repository events into bounded maintenance jobs. One daemon owns a SQLite store in write-ahead-log mode with events, jobs, runs, provider budgets, heartbeats, and temporal blackboard claims. Hooks, filesystem scans, provider interruptions, and explicit commands feed the same store. Stable digests collapse duplicate events; an allowlist limits what may execute; cooldowns smooth provider use; expired owners are recoverable; and a single-resident guard prevents two schedulers from fighting over the queue. Status pages are projections over the store, never repair authority. During one snapshot used for this revision the status surface correctly reported the daemon as not running while queued jobs remained visible. The substrate exists and preserves work, but "always-on architecture" is not the same claim as "currently healthy service".

The operating loop is deliberately plain: observe the current state, classify the task, route it to an owner, claim the work, act, validate, record the result, propagate any reusable lesson, then observe again. "Continuous" work means repeating that loop with explicit checkpoints. It does not mean allowing an agent to mutate indefinitely without a new receipt.

3.5 What continuous mathematical work looks like

A continuous goal is not a request to repeat the same prompt forever. It is a durable research object with a fixed ultimate question, a mutable frontier, and an explicit claim ceiling. Each new run resumes from the latest accepted state: the sources already read, the routes already killed, the computations already interpreted, the formal obligations still open, and the precise statement that remains unproved. The agent then selects the highest-value available transition in that state. It may close a lemma, produce a counterexample, sharpen a reduction, repair a source claim, or expose a smaller obstruction. Merely producing more activity is not a transition.

A representative internal trace makes this less abstract. Over fifteen successive turns, one problem-directed run recorded 313 visible progress updates and 3,491 command events, with linked workers exploring separate analytic, computational, and formal lanes. The movement was not a straight line from prompt to proof. Numerical probes exposed structure; exact arithmetic replaced promising samples; several attractive strengthenings were killed by counterexamples; partial inequalities were narrowed to their valid domains; Lean targets were attempted only after an ordinary mathematical kernel had stabilised; and successful local results were propagated into the problem frontier before the next search began. While long exact computations ran, the controller advanced independent work instead of repeatedly polling them.

That trace also displays why the architecture keeps several evidence layers. Its compact narrative reported no observed changed paths even though its own turn summaries referred to landed commits. This is not evidence that nothing changed, nor that the summaries were correct. It is evidence that a compressed trace has an observation boundary. Repository state, command results, diffs, formal builds, and commit objects must settle the question. Likewise, four turns lacked complete closeout events and only sixteen of forty-two linked session outcomes appeared in the compact projection. Completeness is therefore an explicit field, not an impression created by fluent prose.

The example is illustrative rather than a throughput benchmark. It shows the control shape actually used: search and validation interleave; failed routes remain informative; concurrency is conditional on separable work; and every claimed advance must eventually cross from a narrative of activity into an authority-bearing artefact and receipt. The design aim is continuous mathematical pressure with discrete, inspectable commitments. A run may stop because the next step needs an unavailable validator, a human decision, or a new idea. It must then preserve a useful re-entry point rather than manufacture busywork or quietly weaken the endpoint. Long-running computation is likewise not a reason to poll idly when an independent proof, source, exposition, or adversarial lane remains safe to advance.

3.6 Coupled continuous goals: discovery and stewardship

One continuous goal is not enough for a moving mathematical corpus. A problem-directed miner can remain close to the proof frontier, yet that same proximity makes it a poor sole judge of whether a new lemma is routine, whether an older theorem is stronger, or where the result belongs in a paper. The implementation therefore admits two coupled goals with different responsibilities. The *discovery goal*

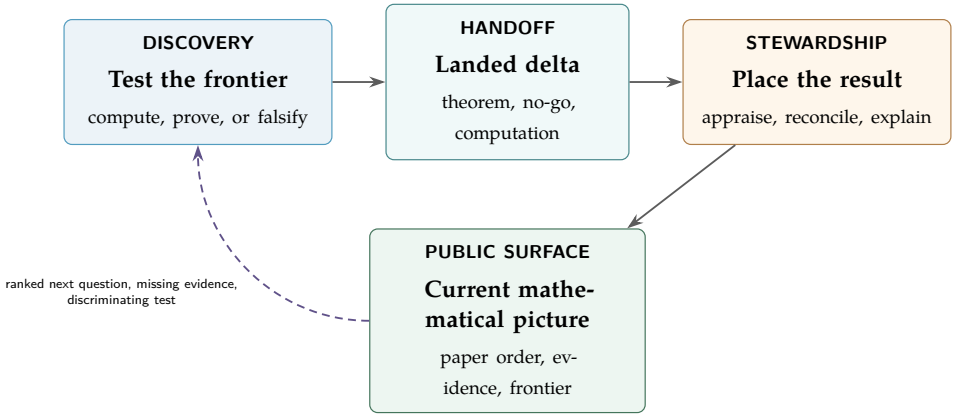

Figure 3: The coupled continuous-goal lifecycle. A landed mathematical delta wakes stewardship; a changed appraisal or consumer gap can change the next mining target. An unchanged corpus produces no model heartbeat.

reads the current problem world, runs discriminating computations, attempts proofs, records falsifiers, and returns the smallest stable mathematical delta. The *stewardship goal* reads that delta against the whole source-current corpus, composes related declarations into coherent result families, and reconciles their downstream uses. Neither role receives the other's authority.

The stewardship pass makes four decisions separately. *Authority* records what Lean, a computation, a paper argument, or an external source actually establishes. *Mathematical appraisal* asks about logical reach, mechanism depth, independence, sharpness, reuse, and the surviving open boundary. *Exposition placement* determines what leads the abstract, which theorem receives the longest explanation, and what remains subordinate. *Work allocation* determines which missing implication or consumer deserves the next unit of compute or expert attention. A result can be highly significant but mechanically unready, fully checked but mathematically routine, or ideal as a worked example without being the corpus's strongest theorem. No single scalar or status is allowed to decide all four questions.

This is also a second checking layer, but not a second proof authority. The steward may detect that a purported advance merely restates the open problem, that several theorem names express one result, that a paper has buried a stronger theorem, that Comparator exposes a weaker interface, or that a Palomar packet omits the hard mechanism. It may then narrow, regroup, reorder, or defer the projection. It may not turn significance, repeated agreement, or polished exposition into a proof. Lean validation, intended-meaning review, prior-art assessment, and community acceptance remain distinct gates.

The clone-local `run-coupled-research-goals` skill operationalises this division by invoking `mine-open-problem` and `propagate-research-consequences` as separate jobs with source-pinned hand-offs. A single agent may execute the two jobs sequentially; two tasks, machines, models, or people may also hold the roles. The separation is logical, not a requirement to buy two computers.

The coupling is event-driven. A newly landed theorem, counterexample, authority change, paper correction, or external review outcome can wake the stewardship goal. Its output is a source-pinned appraisal and a changed consumer or frontier disposition, not a recurring status report. That output may in turn wake the discovery goal with a stronger target, a missing hypothesis, a counterexample request, or the cheapest discriminating computation. If neither the mathematical frontier nor a downstream consumer has changed, both goals yield rather than polling one another.

Before either role begins, `explain-public-system` provides a separate cold-clone orientation job. An agent can read the public corpus and companion papers, adapt the explanation to a lay reader, mathematician, formaliser, compute contributor, reviewer, or infrastructure contributor, and point to the exact evidence behind its account. The human therefore need not learn the file layout before asking a useful question. Explanation remains a projection: it cannot promote its own summary into proof or acceptance.

4 The mathematical reasoning loop

The mathematical lane begins before Lean. A problem is first stated in ordinary mathematical language, with its known status and source. Candidate mechanisms are then generated and ranked. Rank is based on mathematical signal—endpoint proximity, depth of mechanism, independence from other routes, usefulness for future work, and overclaim risk—rather than theorem count or ease of formalisation. Attention is deliberately unequal: a decisive obstruction or a close conditional reduction may deserve more work than many routine lemmas.

4.1 Experiments are route selectors

Computation serves three useful roles. It can find a counterexample, measure a finite pattern, or test whether a proposed mechanism is plausible enough to formalise. Each experiment records its inputs, code, finite domain, output, and interpretation. The interpretation must state what the experiment cannot show.

A finite computation becomes formal evidence only through an explicit bridge. For example, Lean may evaluate a finite proposition with `decide` and check the resulting proof term. Even then, the conclusion remains finite. A pattern observed for many inputs does not become an "all inputs" theorem, and a successful numerical approximation does not become an exact equality. Failed experiments are kept when they prune a natural route; otherwise later agents pay to repeat the same mistake.

Failure modes are mathematical outputs. The system distinguishes three kinds of negative evidence. A failed agent attempt says only that one search path did not close. A counterexample can refute a conjecture on its exact domain. A Lean no-go theorem can rule out a whole strategy class under explicit hypotheses. The last two may be as valuable as a positive lemma: they prevent repeated work and reveal which new ingredient an endpoint requires. This corpus therefore keeps deep problem-specific reasoning surfaces, proof gaps, coefficient-only and fixed-precision obstructions, and corrected statements beside successful theorems. Every no-go keeps its scope visible; failure of one mechanism is not failure of every possible proof.

4.2 Three oracles, not one

Experiments are route selectors; they do not become a fourth oracle. The workflow separates three answers that are often compressed into the word "verified".

1. A *status oracle* says what a public registry or the literature currently reports about the problem.
2. A *formal oracle* says whether the pinned Lean environment accepts this exact statement and proof under the permitted axioms.
3. A *research-validity judgement* asks whether the formal statement matches the intended problem and whether the result matters in context.

The second can be mechanical. The first can become stale. The third remains a mathematical and scholarly judgement. A successful Lean build without statement reconciliation is therefore not the end of the reasoning process.

The learning loop grows the corpus it reasons over. An agent first navigates the existing Lean and semantic graph, then uses proof search and bounded computation to probe promising gaps and patterns that may be too distributed for an unaided manual scan. The checking pass records the kernel result, reconciles formal and informal statements, and packages either a theorem, a counterexample, or a diagnosed failure. Accepted nodes and edges improve the next search; reusable process lessons improve heuristics and context, never mathematical truth. When a mechanism comes from the literature, its annex record, locator, and public citation preserve the original authorship and claim scope while Lean checks only the new formal statement. Thus the cycle is: ground sources, navigate, compute, conjecture, formalise, interpret, publish, and return the enlarged graph to the next iteration.

4.3 From local progress to reusable mathematics

Problem-sized worlds are deliberately local, but their useful by-products need not remain local. A lemma discovered while attacking one Erdős problem may encode a comparison principle, finite combinatorial device, analytic estimate, or Lean interface that belongs in a wider mathematical library. The system therefore distinguishes *process up-propagation*—a lesson that improves how later agents work—from *mathematical canonicalisation*—the conversion of a local result into a reusable theorem.

Canonicalisation is a second research act, not a change of filename. It asks which hypotheses the proof really uses, removes problem-specific coordinates, searches for prior art and existing library interfaces, and states a natural general theorem for which the motivating local result is an explicit specialisation. The candidate is then attacked again: dropped hypotheses need counterexample tests, the general proof needs its own exact check, and at least one genuine additional consumer or explanatory use should be identified. Broader syntax without broader mathematical use is not yet a canonical result.

This yields three statuses that must not be collapsed: checked inside one problem world; proposed as reusable mathematics; and suitable for an external shared library. The present system can preserve the first, help construct and test the second, and prepare evidence for the third. It cannot grant the third status to itself. Mathlib is open to new contributors, but its current guide sets high standards for generality, integration, maintainability, style, documentation, and responsible disclosed use of AI [13]. A subject and Lean expert must decide whether a candidate belongs there, reshape and review it as needed, and take responsibility for any upstream discussion or pull request. The originating route should remain visible in provenance; the expert should receive explicit credit for the generalisation, library design, formalisation, review, and stewardship actually supplied. The open-source strategy gives the corresponding participation and credit protocol.

4.4 Problem-sized Lean worlds and bounded theorem neighbourhoods

Large formal corpora need several maps because "everything is indexed" is not the same as "everything is understood." The declaration atlas answers "where is it?"; exact imports and dependencies answer "what does it formally use?"; authored semantic relations answer "what role does it play?"; and the claim graph answers "what may the project publicly say?" Generated or inferred edges can select evidence, but only source-specific gates can supply proof, interpretation, or publication authority.

The scale is already problem-sized. At the August 2026 snapshot the attached public corpus described 1,024 Lean modules and 153,396 declarations. The much deeper private record for Problem 1041 contained 177 exact results, seven open producers, 72 negative results, 134 Lean modules, and 335 executable experiment programs. Its public research-corpus manifest alone recorded 685 files and 35 selected strongest results. These are dated inventory facts, not a throughput benchmark; generated certificate families also make raw theorem counts a poor measure of mathematical value.

Depth is not readiness. The canonical public 1041 library currently has two integrated Lean modules, while its separately governed public research export has 125 Lean sources and its private world is larger again. The public route memory still records no reviewed 1041 claim family. Across the whole semantic corpus, structural linkage is broad, but only 25 statement nodes and eight relations carry digest-bound semantic-review receipts at this snapshot. These differences are intentional status boundaries: a discoverable declaration is not thereby understood, integrated, reviewed, or publishable.

An agent does not receive this world as one prompt. First the problem cockpit fixes the endpoint, status, claim ceiling, and canonical frontier. It keeps open producers distinct from the target so a promising route cannot silently replace the problem. Next a bounded neighbourhood selects the strongest relevant results, exact source coordinates, imports, consumers, alternative formulations, experiments, counterexamples, no-gos, and literature. It labels every item by evidence class and emits an omission receipt with an expansion route. One 1041 packet exposed only four of 134 Lean modules, four of 335 experiments, and four of 55 no-go notes; designed omission, not exhaustive loading, made the context usable. When an exact path is claimed, a local connection card can put its prerequisites, sibling mechanisms, consumers, falsifiers, and validation target before broader retrieval.

4.5 Comprehension before the mathematics

The neighbourhood is compiled, not remembered. Before attempting mathematics an agent asks a working-memory compiler for a packet conditioned on its query, and that packet, rather than the repository, is what it reasons over.

Federation works by identity. Each corpus keeps its own authority and nothing is copied into a central index. The compiler holds a compact descriptor and a content fingerprint for the private projection and for every attached public checkout, and it opens an attached exhaustive atlas only after those identities reconcile and the bounded route has failed to answer. It never rebuilds the expensive Lean microcosm. Every packet names the corpora it consulted and the digests at which it saw them.

The packet leads with the endpoint. A problem cockpit fixes the target statement, its status, the claim ceiling, and the current claim frontier, labelling each frontier claim with an evidence class and a logical altitude that separates an equivalence from a reduction, a necessary condition, a finite exclusion, and a no-go. Beneath it a mechanism landscape keeps proved or kernel-checked results apart from open producers and from routes a counterexample has closed. Retrieval moves between global frontier structure, concept neighbourhoods, and premise or obligation ancestry, and exact imports, authored arguments, generated navigation, lexical references, and semantic inference remain visibly different edge classes.

Above that material sits an inference workspace: an exact commitment plane for source coordinates and statements, a strategy plane whose unit of branching is a coherent premise group, and an adversary plane for the falsifiers a branch must survive. Its commitments are invalidated whenever a corpus fingerprint moves. It states its own boundary in every packet, that only Lean, an exact counterexample, or an owner verifier receipt may change the status of a claim. The workspace plans inference and concludes nothing.

Two refusals are as important as the retrieval. A query naming no theorem, declaration, or claim is reported as unanchored, and its consumer is told to name one before treating any route as target-specific, so a vague question does not receive a specific frontier. A packet over its requested context budget says so and exits non-zero instead of truncating quietly: it reports the requested and estimated token counts, names the planes it dropped, and gives the command that restores them. An omitted adversary plane appears as an omission with an expansion route, never as an absence of falsifiers.

This is orientation, not understanding. A packet can be well formed over corpora it identified correctly and still select a weak mechanism, omit the governing obstruction, or carry an authored interpretation that is mathematically wrong. The supported claim is narrower: an agent entering a corpus of this depth receives a bounded starting state that names its sources by digest, labels its evidence, and declares its own omissions, and no part of that state carries proof authority. Whether agents working from such a packet produce better mathematics than agents without one is not measured here.

4.6 Consequence propagation

When a declaration lands, the direction reverses. A consequence mapper begins at the exact changed object and enumerates reverse imports, claim records, validators, experiments, papers, and open obligations that may now be stale. A semantic second pass must choose: update now, verify unchanged, defer with a reason, or mark outside scope. Empty lexical search is not evidence of no consequence, and no projection may bulk-strengthen a family of claims. A formal evidence cell carries the result, its explicit non-claim, evidence class, source and receipt links, and next unresolved obligation through this fan-out. The public `propagate-research-consequences` skill implements the same discipline without depending on the private workbench. For work returned from an older clone, it runs once against the contributor's recorded starting commit and again after the reviewed change has been reconciled with current main. The original delta and any conflict-resolution delta remain separate evidence and receive separate attribution.

This same architecture distinguishes mathematical propagation from process propagation. A theorem, counterexample, or no-go changes the mathematical graph only through its own verifier. A reusable failure in navigation, scheduling, experimentation, or validation enters a guarded packet containing the local case, failure class, evidence, sibling scan, proposed owner, overgeneralisation guard, validation, and stop condition. If accepted, it may change a route, skill, check, or standard for later agents. The mechanism is implemented and has worked examples, but there is not yet a complete mea-

sure of what fraction of useful failures propagate. The supported claim is that failures *can* alter the durable control plane without altering mathematical truth.

5 The public Lean repository

The public checkout begins at a deliberate boundary. It contains every file needed to inspect its claims and replay its checks. It does not call back into the private workbench, and no public theorem depends on an unpublished private lemma. The larger workflow is provenance: it explains production, not truth.

The repository has two Lean roots. The reviewed root contains the established 249/257 publication lane. The problem-owned expansion root contains work on six additional Erdős problems and unpromoted lanes for 249 and 257. Both roots are checked by the same Lean kernel, but kernel acceptance does not automatically promote a declaration into a reviewed public claim. Promotion is a separate change to the claim record and exposition.

The main public sources have deliberately separate jobs:

Surface	Authority	It does not establish
Lean source and pinned toolchain	Exact formal statements and proofs accepted by the kernel.	Intended meaning, novelty, significance, or faithful prose.
Reviewed claim record	Approved wording, status, evidence, bounded domain, and adjacent open statement for selected claims.	Correctness or completeness of the human review.
Papers and guides	Human explanation and reading order within the recorded claim ceiling.	New formal authority.
Generated maps and query tools	Bounded navigation across declarations, problems, graphs, papers, and claims.	Proof or permission to strengthen a claim.
Release checks and continuous integration	That configured identities, relationships, generated files, licences, and negative tests pass on a named revision.	Understanding of unrestricted prose or independent mathematical approval.

Table 3: The public authority split.

The companion public Plectis repository has a different role. It publishes runnable, bounded mechanism slices and receipts from the wider system. The Lean repository publishes a mathematical corpus. Neither public repository inherits authority from the other, and neither is a public mirror of the private root.

6 Comparator, Palomar, and publication

6.1 Comparator: an exact-statement firewall

Comparator protects a narrow but important boundary. Selected propositions are declared again in a challenge module without their proofs. A solution module must provide terms of those exact types. The configuration pins the permitted axioms, the modules, and a runtime receipt. A named altered statement must fail, which checks that the harness has not become vacuous.

Comparator therefore answers: "Does the proof-bearing corpus still implement this separately stated Lean interface under this axiom budget?" It does not answer whether the interface is the right translation

of an informal problem, whether the theorem is new or important, or whether an independent human has reviewed the proof. "Comparator-checked" is accurate; "independently verified" is not.

6.2 Review selection, and what the Palomar registry is not

Comparator's roster is an evidence inventory, not a ranking. A local review-selection layer adds the missing editorial step. It groups declarations into result families, ranks them by mathematical signal, keeps the hard mechanism and surviving boundary adjacent, and selects a small review portfolio. A qualification check can say that the local packet satisfies its structural requirements. Its records sit under a Palomar-named showcase file because selected families may then be submitted to the external Palomar registry of Lean-verified mathematics. Palomar's own documentation describes mechanical proof checks and automated editorial filtering; it performs no human peer review and does not rank significance. Local qualification, registry inclusion, and human mathematical review are different events. None, by itself, establishes broad acceptance or significance within the mathematical community.

This separation matters under proof abundance. Counting theorems rewards generated volume and routine closure. The local selection instead asks which result changes the mathematical picture, which conditional route is closest to an endpoint, which obstruction prevents wasted work, and which explanation will help an expert assess the claim. The ranking may guide attention; it cannot alter proof status.

6.3 Publication and propagation

After formal proof and statement reconciliation, a result may enter an authored paper, a claim record, a Comparator packet, and a local review unit. Each is a separate projection with a separate ceiling. A public release is made only after the Lean build and the release-surface checks pass, and publication remains a human action.

The return path is equally important. A proof failure may improve a formalisation heuristic. A counterexample may close a family of tempting routes. A reviewer objection may require a narrower public claim. A release drift may create a stronger check. Those lessons propagate to the smallest durable owner—a theorem, experiment record, skill, standard, route, or claim boundary. They do not rewrite raw intent, generated views, or mathematical status by implication.

6.4 From formal checking to mathematical use

Tao argues that AI mathematics should be judged along a chain: proof generation, verification, exposition, publication and community digestion, then eventual canonicalisation [12]. This architecture is a concrete attempt to build for that whole chain. The private reasoning and experiment loops support generation. Lean checks formal proofs, while Comparator protects selected exact interfaces. The problem papers, reasoning surfaces, graphs, and claim records support exposition. Local review selection triages scarce expert attention, and the contribution and release paths support attributable publication. Digestion, acceptance, and canonicalisation remain achievements of the mathematical community, never local status fields.

This mapping also explains the unusual emphasis on failure. Tao warns that AI-polished exposition can erase the natural friction that tells a reader where the real mathematical difficulty lies. The reasoning surfaces and no-go graph preserve that friction deliberately: false starts, corrected claims, missing producers, and formal obstructions remain adjacent to the successful theorems. The purpose is not to display internal noise. It is to transmit the shape of the problem so another mathematician can understand why the surviving boundary is hard and where a genuinely new idea must enter.

Paper authoring itself participates in this loop. As the mathematical and control graphs change, agents assemble and revise problem papers, architecture papers, reviewer cards, and claim records from the source-current evidence. Writing is therefore an active digestion and interpretability pass in Tao's sense: a missing explanation, unstable term, hidden quantifier change, or unregistered limitation discovered while writing becomes a typed defect that can return to the theorem, semantic relation, claim record, route, or validator. This manuscript is a reflexive example: it was rewritten by the system while agents inspected the mechanisms it describes, then compiled and checked as a public artefact. That reflexivity is provenance, not validation. A system does not independently review itself merely by producing a clear account of itself.

The remaining design choices follow the same programme. Tool use is disclosed; models are not listed as authors; human contributors retain differentiated credit; references and source paths are inspectable; automatic checks filter work without impersonating peer review; and theorem counts do not determine value. The repository turns those recommendations into an implementation experiment: it asks how a small project can prepare for proof abundance without confusing abundant output with collective mathematical progress.

7 One complete boundary: finite is not unbounded

Erdős Problem 249 asks whether

$$S = \sum_{n \geq 1} \frac{\varphi(n)}{2^n}$$

is irrational, where $\varphi(n)$ is Euler's totient function [2]. The development reduces irrationality to a family of exact non-integrality certificates. Along the diagonal $H_t = \operatorname{lcm}(1, \ldots, t)$, write $\mathrm{Cert}(t)$ for the existence of the required finite certificate.

Lean checks the finite theorem

$$\forall t \leq 82, \quad \mathrm{Cert}(t).$$

The endpoint needs an unbounded supply:

$$\forall T, \quad \exists t > T, \quad \mathrm{Cert}(t).$$

The development proves that the unbounded statement is equivalent to the irrationality claim. It does not prove the missing implication from the finite range to the unbounded statement. No matter how large a fixed checked bound is, a larger cutoff exists.

This example passes through every layer. Computation constructs finite data. Lean checks the certificate predicate and the finite theorem. The formal graph records its dependencies. The semantic graph identifies its role. The claim record states the finite range and names the unbounded requirement as open. The paper explains why the quantifiers differ. Comparator checks selected exact interfaces. The local review selection may rank the result family for review. The release checker requires the limitation to remain visible.

The last check was added because the boundary once escaped. A historical README edit changed a clause saying that the finite cases did *not* supply the open requirement into one saying that they completed it. Lean was untouched, and the release checker passed because that prose relationship had not been registered. After the relationship was added to the claim record, a deliberately false copy was rejected. This is evidence for one failure and repair. It is not a detection rate, a proof that every claim-bearing sentence is registered, or an evaluation of mathematical review quality.

The historical study was narrower than a benchmark. In that study, nine of the ten edits were rejected. One escaped because the relationship had not been registered. The original run logs were not retained. The edits were authored by the checker's author. After the relationship was registered, only the escaped edit was reconstructed against the repaired checklist. The other nine edits were not rerun against the extended checklist. The finite theorem makes no $t = 83$ or cofinal claim. These limitations are part of the evidence, not footnotes to be discarded after the repair. The evidence marks a coverage boundary, not a reliability score. The post-repair witness accepts the current README and rejects a test copy containing the false clause.

The same architecture handles other shapes of progress. Problem 257 separates a theorem for full-support representations from a stronger arbitrary-support statement that remains open. Problem 68 has an exact carry-based equivalence but no theorem producing the required carries. Problem 251 has an exact series reformulation rather than an irrationality proof, and Problem 243 records a conditional recovery theorem whose premises remain part of the public claim. A counterexample or corrected statement, as in the 1041 lane, is also a first-class result. The publication rule is identical in every case: preserve the quantifiers, premises, and nearest stronger open statement.

8 Inspection routes

A reader can inspect the public system through short, question-shaped routes. The labels below link to repository paths; the paper does not print the full URLs.

Question	Start here
How does the public repository fit together?	`ARCHITECTURE.md`
What may the project say, and what remains open?	`docs/claims.json` and `docs/methodology.json`
Where is the checked mathematics?	`shared mathematics library` and `problem-specific mathematics library`
How can I navigate without reading the whole corpus?	`docs/ORIENTATION.md` and `scripts/query_corpus.py`
What exactly does Comparator check?	`docs/EXTERNAL_VERIFICATION.md` and `verification/comparator.json`
What does the local review selection qualify?	`docs/PALOMAR_QUALIFICATION.md` and `docs/PALOMAR_RESULT_SHOWCASE.json`
Which papers exist and what question does each answer?	`docs/papers/README.md`
Which checks gate a release?	`scripts/check_release.py` and `.github/workflows/lean.yml`
How can work return with public credit?	`CONTRIBUTING.md` and `research-commons/CONTRIBUTIONS.md`

Table 4: Compact public inspection routes. These are entry points, not a new authority layer.

9 What can be trusted

The architecture supports a chain of narrow conclusions:

1. a recorded experiment ran on its stated finite inputs;
2. a pinned Lean kernel accepted its stated formal source;
3. a maintainer approved a selected interpretation and public boundary;
4. Comparator matched selected proof-bearing declarations to separately stated interfaces and an axiom budget;
5. the local review-selection checks validated the structure of a selected review packet;
6. the release program found every configured public relationship intact on the named revision.

Joining these receipts improves traceability, but it does not create a seventh, stronger oracle. In particular, the architecture does not establish that the formalisation is the uniquely right one, the review is independent, the result is novel or significant, the registered boundary is complete, or an open Erdős problem is solved. A coordinated but mistaken edit to source, record, and prose can still make every structural comparison agree. Unregistered prose can still overclaim. Literature status can still change.

The system reduces these risks by keeping sources plural, boundaries explicit, and negative tests close to high-risk claims. It does not claim to eliminate human judgement. It does not technically force a

second independent mathematician to review the result. This is the same limit encountered in assurance cases: a formal notation can record an asserted support relationship without proving that the evidence is true or sufficient for the top claim [11].

A reader can keep those limits straight by asking, for any capability named in this paper, which of four kinds of support stands behind it.

Kind of support	Example	What may be said
Executable check	A session or return validated against required fields and source identity; a Comparator interface match	The program checks the stated conditions on the stated inputs
Agent workflow instruction	The discovery and stewardship responsibilities in a public skill	The workflow directs the agent; it does not show that every agent complied
Mathematical judgement	Whether a result expresses the intended mathematics or improves on prior work	A named assessment is required; agreement between source, record, and prose is not one
External outcome	Another researcher reproduces, adopts, corrects, or builds on the work	A recorded external event is required; the architecture implies none

10 Scaling from one clone to a search network

10.1 Clone, attach compute, and mine bounded results

The public repository is more than a static archive. A fresh clone fixes a Lean toolchain, exposes the supported roots, names reviewed claims and open boundaries, and provides query and release programs. An external researcher can attach any agent runner to those public interfaces, launch independent workers on disjoint theorem or experiment lanes, and submit candidate changes through the same proof and publication gates. More compute increases the number and diversity of attempts; it does not relax the meaning of a passing receipt.

This suggests a "result mining" mode. Workers select bounded frontier objects, explore proofs, counterexamples, computations, or literature, and return exact artefacts. Fan-in then checks Lean, reconciles statements, records negative evidence, and ranks the mathematically strongest surviving result families. The useful unit is not a model answer or a theorem count. It is a result packet with provenance, scope, formal status, mathematical role, and a nearest stronger open statement. Local review selection can allocate scarce expert attention to those packets; Comparator can protect selected exact interfaces; neither is replaced by scale.

At scale, fan-in is a persistent stewardship role rather than an end-of-run cleanup task. It consumes each stable delta, compares it with the complete candidate universe, updates the paper hierarchy and assurance interfaces when warranted, and returns a newly ranked frontier to the mining fleet. Several miners may therefore share one stewardship goal, and one steward may consume results from several problems, provided every write and mathematical object still has one owner. The steward is allowed to say that a large run produced no new mathematical family, or that a short negative result should outrank many formal lemmas. This prevents compute volume and theorem count from becoming accidental editorial policy.

Fleet size and validation capacity are deliberately different variables. In principle, many logical agents can search disjoint mathematical neighbourhoods; in practice, the controller admits work according to path conflicts, memory, CPU, disk pressure, and the expected value of the lane. Cheap reading and computation may fan out widely while mutation remains path-leased and heavy Lean work is narrower. This prevents "more agents" from meaning "more processes contending for the same compiler and checkout."

Lean validation uses a semantic single-flight queue. A request is identified by the reachable source, logical targets, toolchain, dependency lock, and authority mode. Its command string and working directory do not determine request identity. Equivalent requests share one owner and may reuse a completed receipt; changed inputs require a fresh build. Distinct requests that would still compete for the same host-wide Mathlib resource are serialized by a shared conflict key. An interactive agent that cannot acquire the slot receives a typed deferral, not a false theorem failure: it can advance result-independent work while a detached process owns the queued build. Thus the fleet may grow much wider than the expensive validator without corrupting the evidence or wasting every agent as a queue supervisor.

There are consequently four separate scaling limits: logical search width, safe concurrent mutation, mechanical validation throughput, and scarce expert review. Path leases govern the second, the Lean queue governs the third, and review-selection ranking governs the fourth. Fan-in joins their outputs only after the appropriate receipts exist. This separation is what allows additional models and compute to enlarge exploration without silently enlarging any model's authority.

The return path is already public. A person can fork or clone the repository, work from a named public commit, and open a pull request or research-progress issue. A structured return can preserve the contributor, collaborators, tool operator, disclosed model systems, starting commit, evidence, affected result, and surviving limitation as separate fields. Only an accepted receipt enters the generated contribution views. Acceptance, mathematical claim status, and release inclusion remain separate decisions, so a credited counterexample or failed route need not be mislabelled as a theorem. Later corrections append a new history rather than erasing the earlier contributor. Attribution and pull requests are standard practice; the architectural role here is to carry credit and provenance through fan-in without allowing either to strengthen the claim.

The named starting commit remains useful even when main advances. Git retains the common ancestor needed to inspect the contributor's original delta. A maintainer can replay that state, reconcile the reviewed substance with the current tree, and rerun current validation and consequence propagation. A material conflict resolution is a new integration contribution; it does not rewrite the origin of the earlier work.

The exact contributor-facing sequence is specified in the companion contribution protocol; its credit model is separated in the provenance and credit section. The navigation assumptions made here are tested by the cold-clone case study.

The current public clone supplies the mathematical corpus and its gates. The full private orchestration layer is not yet distributed as a turnkey public service, so mass multi-provider mining is a design target rather than a reported benchmark. The important point is that the public interfaces do not depend on one private scheduler. A laboratory with many frontier models can replace the producer while retaining the same evidence boundary.

No outside contributor had completed this path by 31 August 2026. The cross-paper links, clone-local skills, and pull-request route are therefore implemented prototype interfaces whose usability remains an external test.

10.2 The no-go graph as a new mathematical object

A mature corpus should not represent progress as a list of proved theorems. Its graph should contain at least the following node classes: endpoint problems; conjectures and reformulations; proposed mechanisms; computations; formal lemmas; counterexamples; no-go theorems; reviewed claims; and unresolved obligations. Edges should distinguish implication, equivalence, dependency, refutation, obstruction of a strategy, weakening, generalisation, experimental support, formalisation, and publication. A no-go then occupies a precise place: it closes one region of mechanism space while exposing the boundary around nearby variants that remain viable.

As this negative and positive graph grows, several new uses become testable. A model may navigate around already closed strategy families, identify a small cut of missing premises between the current corpus and an endpoint, or search for analogies between obstruction patterns in different problems. Training data can be formed from proof/counterexample/no-go triples or from contrastive pairs consisting of a tempting argument and the exact theorem explaining its failure. Graph-conditioned models might propose the next useful lemma from a frontier neighbourhood rather than from the theorem statement alone. These are hypotheses about future systems, not results established by this paper.

Negative structure also creates risks. An over-broad obstruction edge can hide a viable variant; repeated failed attempts can teach stylistic avoidance rather than mathematics; and model-generated semantic edges can disagree with the formal graph. Training snapshots therefore need immutable generations, source-level provenance, explicit edge authority, and held-out evaluation. Lean can verify formal nodes and some edges, but expert judgement remains necessary for semantic roles and for deciding whether the graph captures the important mathematical space.

10.3 An experimental agenda

The next evaluation should ask more than how many theorems additional compute produces. It should compare graph-aware and graph-blind agents on frontier selection, repeated-dead-end rate, time to a useful obstruction, premise discovery, proof completion, and calibration of public claims. It should test transfer between unrelated mathematical domains as well as between neighbouring Erdős problems. Independent teams should replay result packets and review whether no-go edges are scoped correctly. The central scaling question is whether an increasingly rich map of what works and what cannot work makes future reasoning more efficient and more original, or merely makes the system more confident about the territory it already knows.

11 Relation to other approaches

Theorem-proving agents. This architecture is not a new proof-search algorithm. LeanDojo and Pantograph provide programmatic Lean environments and proof-state interaction [14, 15]. OpenProver already combines a planner, parallel workers, independent verifiers, compact working memory, a larger repository, and Lean feedback [16]. Agent Hunt already studies concurrent formalisation with locks, bounties, guarded ownership, and collaborative agents [17]. DreamProver learns a compact reusable lemma library through wake–sleep cycles [18]. These are baselines for proof interaction, parallel search, and learned proof memory. Here those mechanisms sit upstream of a different question: after a proof is found, what exactly may move into a reviewed public claim?

Graphs and checked exposition. Proof blueprints connect informal proof plans to named Lean declarations. `leanblueprint` checks that author-supplied declaration names exist, while LeanArchitect infers formal dependencies and unfinished-proof status and exports synchronised blueprint material [3, 4]. The graph layers here have a related navigational role, but the public-claim record begins after a result has been selected and asks which wording and open boundary were reviewed.

Semantic-audit systems address a neighbouring problem. Lean Atlas narrows the declarations a person must inspect for chosen theorem statements, conditional on the semantic correctness of the returned set and trusted base [5]. EconCSLib uses models to translate and compare formal and informal statements, with saved human judgements for a subset [6]. The present architecture uses models throughout production but assigns none of them final semantic authority. Their output becomes evidence or a candidate until a source-specific gate accepts it.

Checked-document and traceability systems make heterogeneous relationships explicit. Isabelle/DOF places formal and informal material in a typed checked document [7]; requirements traceability follows commitments across development and revision [8]. This repository keeps prose unrestricted and records selected public boundaries separately. That choice makes adoption simple, but leaves unregistered prose outside the checker.

Auditable scientific agents. HEP makes hypotheses, evidence, belief updates, lineage, and resolution states explicit in an append-only registry [19]. Symposium proposes immutable community publication histories with attributable artefacts, declared evidence, assumptions, and purpose-sensitive arguments [20]. EurekAgent treats permissions, artefacts, budgets, and human supervision as first-class parts of an agent environment [21]. Persistent records and environment design are therefore not unique to this system. The narrower distinction here is between authorities that must not be collapsed: experimental evidence, kernel acceptance, reviewed interpretation, exact-statement assurance, editorial selection, and public release.

Mutation testing asks whether a test distinguishes a seeded fault from the original program [9, 10]. The deliberately false boundary in the worked example follows that idea. Because the examples were selected

by the system's author and were not run as a controlled external evaluation, they show that particular checks can fail; they do not measure overall adequacy.

Contribution and limits. The contribution is architectural composition, not a claim that each component was invented here and not evidence of better proof-search performance. The system joins a private authority-aware workbench to a self-contained public Lean repository, then keeps formal dependencies, semantic interpretation, reviewed claims, Comparator interfaces, review selection, and release relationships visibly separate. It also carries deeply investigated failure modes and formal no-gos through the same public path as positive theorems, without discarding the stronger endpoint that survives. Failure logging and hypothesis refutation have precedents; among the systems compared here, the distinctive contribution is their integration with formal obstruction results, claim ceilings, and publication assurance. The evidence is a detailed design and worked reconstruction from one evolving corpus. It is not a controlled comparison, an independent audit, or a general reliability estimate.

Scaling beyond this corpus. The design is not tied to Erdős problems. A new domain needs stable result identities, source and status records, a formal checker where one exists, and an explicit public-claim boundary; it need not use the same mathematics or even Lean. Larger deployments should federate independently owned corpora rather than build one authority database, separate producer and reviewer teams, benchmark each boundary independently, and preserve immutable result generations. Proof-search layers can adopt distributed task markets, persistent lemma learning, and richer human steering, while publication layers can add external replay, signed review receipts, and cross-project claim links. None of these extensions should allow learned process memory or a graph edge to change mathematical truth by itself.

12 Conclusion

The architecture has two central claims. First, an AI mathematics system must navigate problem-sized mathematical worlds rather than treat a large Lean repository as a bag of files or one prompt. Second, scaling search safely requires reasoning scope, mutation permission, validator capacity, evidence class, public-claim permission, and reviewer attention to remain non-fungible until an explicit fan-in gate. Together these properties make the architecture a claim-transition operating system with a prover as one component.

The boundaries follow immediately. Questions are not work records. Work records are not experiments. Experiments are not proofs. Lean proofs are not interpretations. Interpretations are not public claims. Public claims are not independent review. Review is not community acceptance.

The private workbench makes the early stages durable and concurrent. Type A actors mutate live substrate under claims and checks; Type B actors contribute bounded external reasoning without hidden write authority. Computational experiments prune and prioritise routes. Lean supplies exact formal authority. Graphs make a large corpus navigable without collapsing inventory into meaning. Comparator checks exact statement interfaces. Local review selection directs scarce review attention, and the external Palomar registry checks and filters what is submitted to it. The public repository then packages source, claims, papers, and release receipts into a clone that stands on its own.

The useful result is not an automatic mathematician and not a universal publication verifier. It is a system in which the evidence for a claim, the person or program allowed to judge it, and the stronger statement that remains open are all visible at the point where the claim moves forward. That is what lets an AI-assisted research process scale without turning accumulated output into accumulated ambiguity.

A Reproducibility

Public first contact. A fresh public clone can reproduce the control card and structural checks:

```
python3 scripts/proof_cockpit.py --format card
python3 scripts/proof_cockpit.py --check
```

The first reads committed public metadata; the second checks the claim registry, cold-clone contract, and orientation freshness. Neither checks a proof. Formal authority begins with the pinned Lean build named by the card.

Dated navigation counts. At the semantic review's snapshot, taken before the eight-problem consolidation of 2026-08-02 enlarged the corpus to 153,238 declarations, all 151,761 then-live declarations were inventoried and routed, and all 144,631 author-written theorem-like declarations had an exact node link. Of those, 139,772 (96.6%) participated in authored mathematical interpretations: 3,284 as exact proposition evidence and 136,488 as bounded certificate- or module-family context. The remaining 4,859 were linked only through exact source-module and normalised-signature families, not authored mathematical paraphrases. Every declaration selected for a public claim had an authored route. The command `python3 scripts/query_semantic.py coverage` derives these volatile navigation counts and checks their references. They do not measure semantic review quality or public-claim completeness.

The paper inventory, `docs/publication_contract.json`, records source and PDF cryptographic hashes and validation commands. The worked-example evidence is recorded in `docs/publication_evidence.json`. The reconstruction file `experiments/publication_mutations.json` specifies the deliberately false edits and their limitations. Comparator's public configuration and receipt are named by `verification/comparator.json`; the local review-selection readiness and ranking surfaces are named by the corresponding files under `docs/`. These artefacts identify what was checked. They do not interpret unrestricted prose or confer external acceptance.

The private system is described here at the architectural level because it is production provenance, not a dependency of the public result. Private paths, operator material, unreleased work, and private ledgers are neither required nor granted authority by this paper. The public checkout remains the inspection and replay boundary.

References

[1] L. de Moura and S. Ullrich, *The Lean 4 Theorem Prover and Programming Language*, in *Automated Deduction—CADE 28*, Lecture Notes in Computer Science 12699, 2021, pp. 625–635, DOI.

[2] P. Erdős and R. L. Graham, *Old and New Problems and Results in Combinatorial Number Theory*, Monographies de L'Enseignement Mathématique 28, 1980, p. 61.

[3] P. Massot, *leanblueprint*, plasTeX plugin for Lean formalisation blueprints, 2020, software repository.

[4] T. Zhu, P. Monticone, S. Welleck, and J. Avigad, *LeanArchitect: Automating Blueprint Generation for Humans and AI*, in *17th International Conference on Interactive Theorem Proving*, LIPIcs 382, 2026, pp. 25:1–25:16.

[5] B. Yanahama and A. Sannai, *Lean Atlas: An Integrated Proof Environment for Scalable Human–AI Collaborative Formalization*, 2026, arXiv.

[6] N. Garg, *EconCSLib: AI-Assisted Lean Formalization for Economics & Computation Research*, 2026, arXiv.

[7] A. D. Brucker and B. Wolff, *Isabelle/DOF: Design and Implementation*, in *Software Engineering and Formal Methods*, 2019, pp. 275–293.

[8] O. C. Z. Gotel and A. C. W. Finkelstein, *An analysis of the requirements traceability problem*, Proc. First IEEE International Conference on Requirements Engineering, 1994, pp. 94–101.

[9] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, *Hints on test data selection*, IEEE Computer 11(4), 1978, pp. 34–41.

[10] Y. Jia and M. Harman, *An analysis and survey of the development of mutation testing*, IEEE Transactions on Software Engineering 37(5), 2011, pp. 649–678.

[11] SCSC Assurance Case Working Group, *Goal Structuring Notation Community Standard, Version 3*, SCSC-141C, May 2021.

[12] T. Tao, *Mathematics in the age of AI*, preprint, 2026, arXiv.

[13] Lean community, *Contributing to mathlib*, contributor guide, accessed August 2026.

[14] K. Yang et al., *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models*, NeurIPS 2023.

[15] J. Storrs et al., *Pantograph: A Machine-to-Machine Interaction Interface for Advanced Theorem Proving, High Level Reasoning, and Data Extraction in Lean 4*, 2024, arXiv.

[16] M. Kripner and M. Straka, *OpenProver: Agentic and Interactive Theorem Proving with Lean 4*, 2026, arXiv.

[17] C. E. Brown, C. Kaliszyk, and J. Urban, *Agent Hunt: Bounty Based Collaborative Autoformalization With LLM Agents*, 2026, arXiv.

[18] Y. Zhang et al., *DreamProver: Evolving Transferable Lemma Libraries via a Wake–Sleep Theorem-Proving Agent*, 2026, arXiv.

[19] I. Takahara and T. Mizoguchi, *Toward Auditable AI Scientists: A Hypothesis Evolution Protocol for LLM Agents*, 2026, arXiv.

[20] D. Pratt, *Symposium: Trust via Auditable Records for Communities of AI Scientist Agents*, 2026, arXiv.

[21] A. Xin et al., *EurekAgent: Agent Environment Engineering is All You Need for Autonomous Scientific Discovery*, 2026, arXiv.